Documentation technique ; SupTaskFlow

Introduction

Dans ce projet, j'ai développé une application appelée **SupTaskFlow**, qui est un outil de gestion de tâches basé sur le principe du **Kanban board**.

L'idée est de pouvoir organiser des tâches dans différentes colonnes afin de suivre leur progression. Par exemple, une tâche peut commencer dans **To Do**, passer en **Doing**, puis être déplacée en **Done** lorsqu'elle est terminée.

Le projet est divisé en deux parties :

- Un **backend avec Strapi**, qui sert à gérer les données et créer l'API
- Un **frontend avec React**, qui permet d'afficher l'interface et d'interagir avec les données

Ce choix permet de séparer la gestion des données et l'interface utilisateur, ce qui rend le projet plus clair et plus facile à maintenir.

Schéma relationnel des données

Pour gérer les données du projet, j'ai utilisé **Strapi**, qui permet de créer des collections et de générer automatiquement une API REST.

Les deux collections principales utilisées dans l'application sont **Column** et **Card**.

Collection Column

La collection **Column** représente les différentes colonnes du tableau Kanban.

Par exemple :

- To Do
- Doing
- Done

Chaque colonne possède les attributs suivants :

Collection Card

La collection **Card** correspond aux tâches du tableau.

Chaque carte représente une tâche créée par l'utilisateur.

Collection Card

Champ	Type	Description
id	Integer	Identifiant de la carte
title	String	Titre de la tâche
description	Text	Description de la tâche
column_id	Relation	Colonne à laquelle appartient la carte

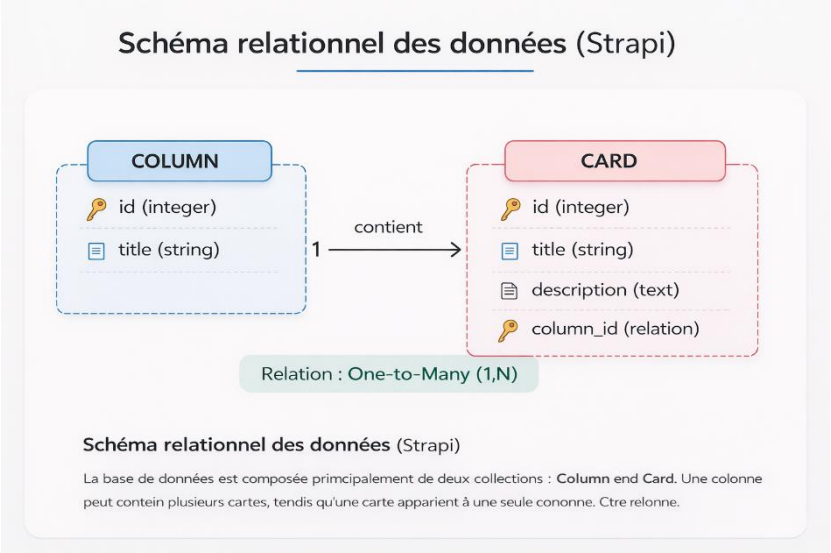
Collection Card

La collection Card représente les différentes tâches de l'application **SupTaskFlow**. Chaque carte possède un identifiant unique (**id**), un titre (**title**), une description, et une référence à la colonne à laquelle elle appartient (column_id).

Relation entre les données

Une **colonne peut contenir plusieurs cartes**.

La relation entre les deux collections est donc :



Cela signifie :

- Une colonne peut contenir plusieurs cartes
- Une carte appartient à une seule colonne

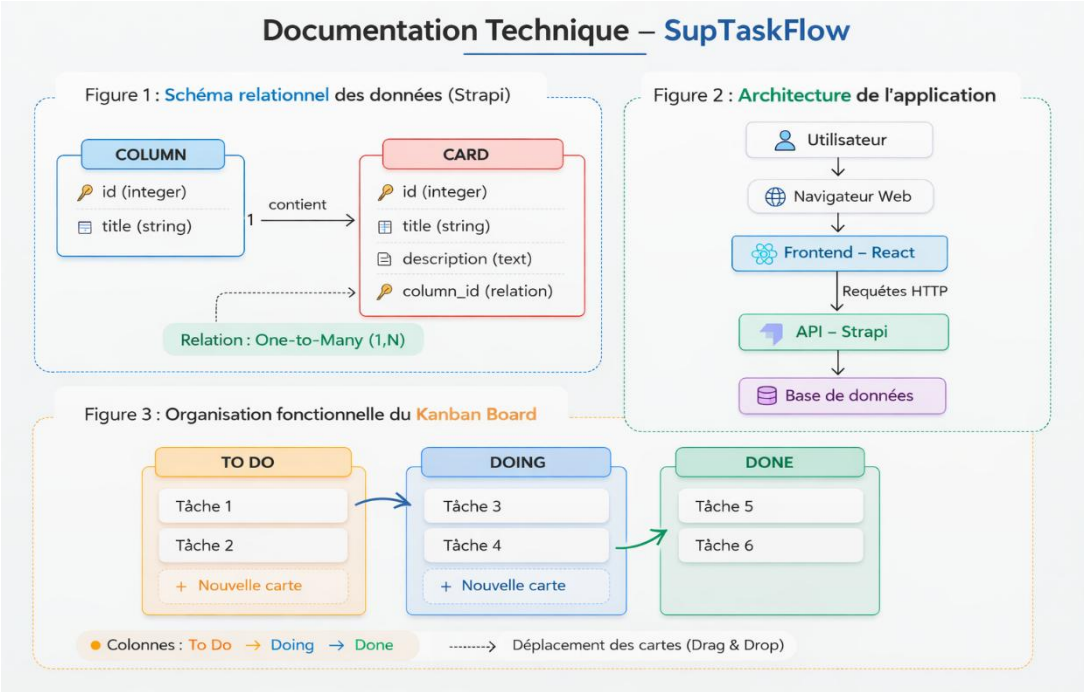
Cette structure permet d'organiser les tâches dans les différentes colonnes du tableau Kanban.

Architecture du projet

L'application SupTaskFlow est basée sur une architecture **frontend / backend séparée**.

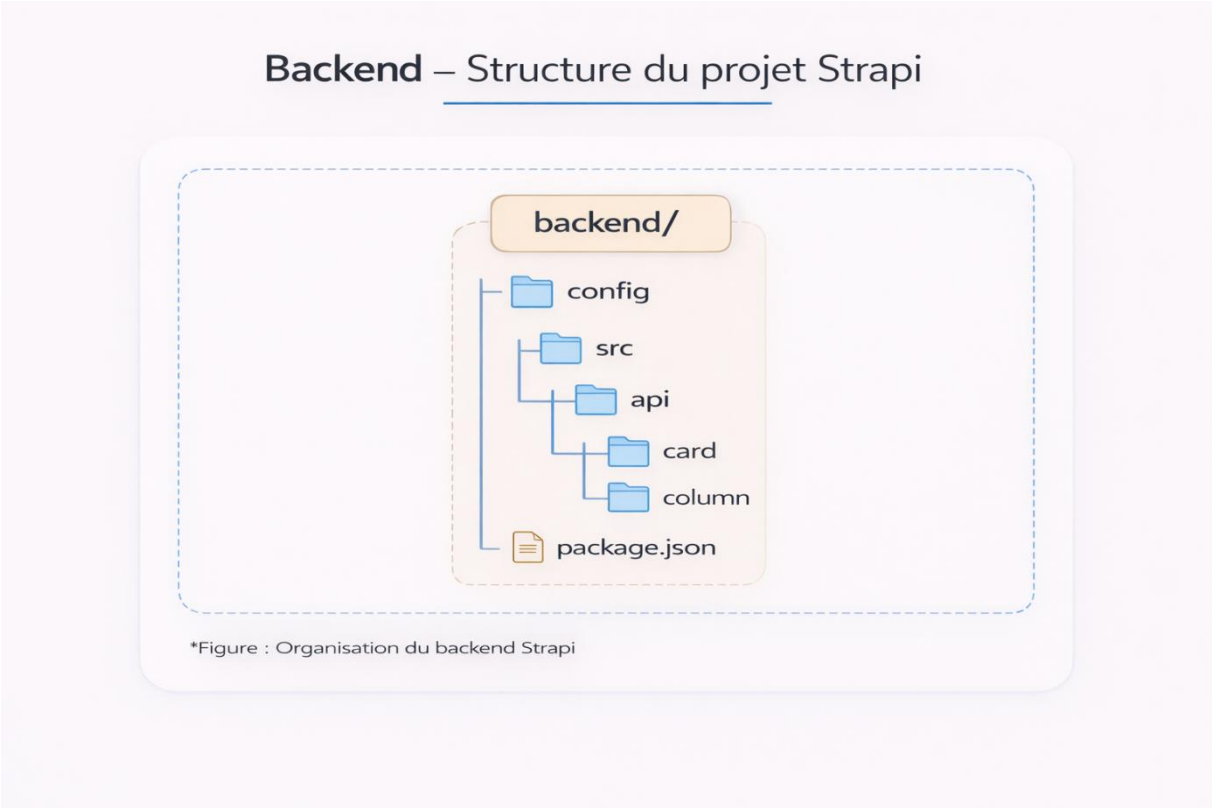
Le frontend est développé avec **React** et le backend avec **Strapi**.

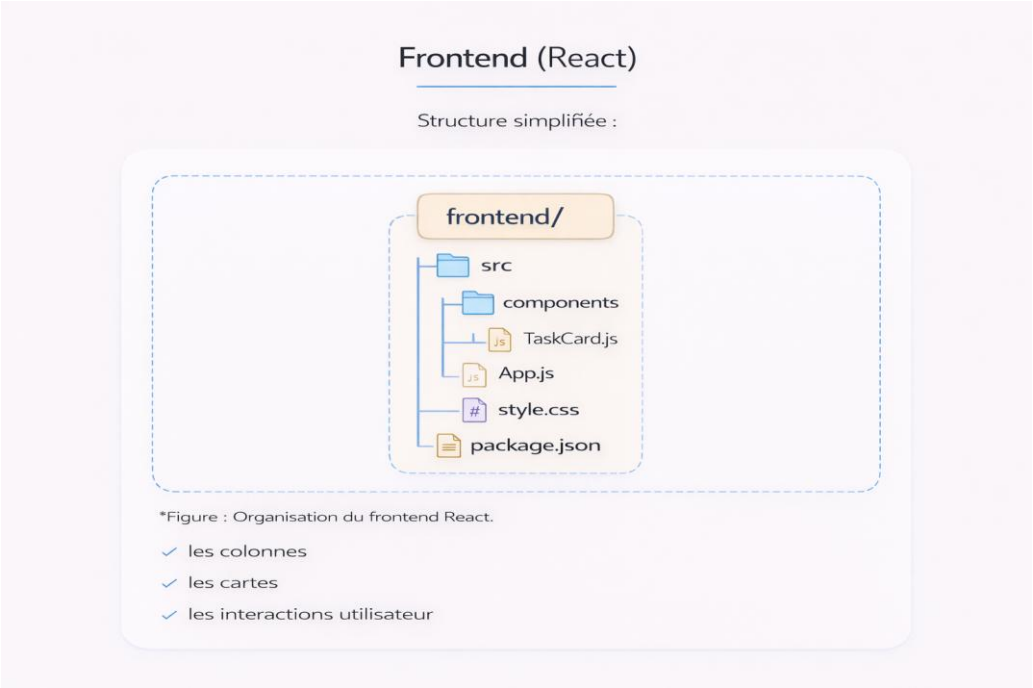
Schéma simplifié de l'architecture :



L'utilisateur accède à l'application via un navigateur web. Le frontend développé en React envoie des requêtes HTTP à l'API Strapi, qui communique ensuite avec la base de données pour stocker ou récupérer les informations.

Le projet est organisé en deux dossiers principaux.





Procédure d'installation

Cette section explique comment lancer le projet sur un ordinateur.

Installation du backend (Strapi)

A. Cloner le projet : Git clone lien-du-projet

B. Aller dans le dossier backend : Cd backend

C. Installer les dépendances : Npm Install

D. Lancer le serveur Strapi : Npm run develop

Le serveur sera accessible à l'adresse suivante :

http://localhost:1337

Lors du premier lancement, il sera nécessaire de créer un **compte administrateur Strapi**.

Installation du frontend (React)

1. Aller dans le dossier frontend :

cd frontend

2. Installer les dépendances :

npm install

3. Lancer l'application :

npm start

L'application sera accessible à l'adresse :

http://localhost:3000

---

6. Variables d'environnement

Le projet utilise un fichier .env pour stocker certaines variables nécessaires au fonctionnement de l'application.

Exemple :

REACT_APP_API_URL=http://localhost:1337

Cette variable permet au **frontend React de communiquer avec l'API Strapi**.

---

7. Choix techniques

Pour ce projet, j'ai choisi d'utiliser **React et Strapi**.

React

React a été utilisé pour développer l'interface utilisateur car il permet de créer des interfaces dynamiques grâce à un système de composants réutilisables.

Il facilite également la gestion de l'état de l'application et l'affichage des données récupérées depuis l'API.

Strapi

Strapi est utilisé comme **Headless CMS** pour gérer les données du projet.

Il permet de :

- créer rapidement des collections
- générer automatiquement une API REST
- gérer les relations entre les données
- administrer les contenus via une interface graphique

Cela permet de gagner du temps dans le développement du backend.

Bibliothèques utilisées

Axios

Axios est utilisé pour envoyer des requêtes HTTP entre React et l'API Strapi.

CSS personnalisé

Le style de l'application est réalisé avec **CSS personnalisé** pour organiser les colonnes et les cartes du tableau Kanban.

Guide utilisateur

Accéder à l'application

Une fois le projet lancé, l'utilisateur peut accéder à l'application via :

http://localhost:3000

Ajouter une tâche

Pour créer une tâche :

1. L'utilisateur sélectionne une colonne
2. Il clique sur le bouton permettant d'ajouter une carte
3. Il renseigne le titre et la description

La tâche apparaît ensuite dans la colonne choisie.

Organiser les tâches

Les tâches sont organisées dans différentes colonnes :

- **To Do** : tâches à faire
- **Doing** : tâches en cours
- **Done** : tâches terminées

Cela permet de suivre facilement l'avancement du travail.

Déplacer une tâche

Les cartes peuvent être déplacées d'une colonne à une autre afin de suivre l'évolution des tâches.

Par exemple :

To Do → Doing → Done
Cela permet de visualiser rapidement la progression des tâches.